MODULE 2: DATA EXTRACTION (09 PERIODS)

Course Code: 22DS102006 | **B.Tech Computer Science & Engineering (Data Science)**

Detailed 10-Page Comprehensive Study Notes: Feature Selection, Decision Trees, Ensembles, Dimensionality Reduction, SVD & PCA.

1. Extracting Meaning from Data

Extracting meaning from data refers to the structured process of transforming raw, noisy observations into informative representations that reveal latent signals, statistical patterns, and actionable domain knowledge. In high-dimensional environments, raw feature spaces contain significant amounts of noise, redundancy, and irrelevant attributes that obfuscate real data relationships.

The core objective of data extraction is signal isolation: distilling complex datasets into minimal, informative feature spaces while preserving maximum predictive power and structural integrity.

Key Stages in Knowledge Extraction

- **Signal vs. Noise Identification:** Separating informative variance generated by true underlying processes from random stochastic fluctuations.
- **Feature Representation:** Transforming domain inputs into numerical vector spaces that respect linear and non-linear manifold structures.
- **Dimensionality Compression:** Projecting high-dimensional spaces onto lower-dimensional manifolds without losing key variance.

Methodological Challenges in Data Extraction

Key challenges include handling correlated features (multicollinearity), sparse feature matrices, high missingness ratios, non-linear feature interactions, and computational scaling bottlenecks when extracting features from large-scale enterprise datasets.

2. Feature Selection Methods

Feature selection is the technique of selecting a subset of relevant attributes from the original feature set for use in model construction. Unlike feature extraction (which creates new features), feature selection preserves original variable interpretations.

1. Filter Methods

Filter methods evaluate features based on intrinsic statistical properties, independent of model algorithms. They operate rapidly and scale well to ultra-high-dimensional datasets.

- **Pearson Correlation:** Measures linear relationships between continuous attributes and target variables: $r = \sum(x - \bar{x})(y - \bar{y}) / [\sqrt{\sum(x - \bar{x})^2}\ \sqrt{\sum(y - \bar{y})^2}]$.
- **Chi-Square Test (χ^2):** Evaluates independence between categorical features and target labels: $\chi^2 = \sum (f_o - f_e)^2 / f_e$.
- **ANOVA F-Test:** Assesses variance ratio between group means for continuous features against categorical targets.

> **Filter vs. Wrapper vs. Embedded Summary**
>
> **Filter:** Fast, model-agnostic, ignores feature interactions.
>
> **Wrapper:** Highly accurate, computationally expensive, prone to overfitting.
>
> **Embedded:** Balanced efficiency, algorithm-specific, captures feature interactions naturally during training.

3. Wrappers, Embedded Methods & Implementation

2. Wrapper Methods

Wrapper methods treat feature subset selection as a search problem, utilizing a predictive machine learning algorithm to score candidate feature combinations based on cross-validation performance.

- **Forward Selection:** Begins with zero features, iteratively adding the feature that gives the highest model performance boost until no further gain is made.
- **Backward Elimination:** Starts with all features, iteratively removing the least significant feature per round.
- **Recursive Feature Elimination (RFE):** Fits a model and recursively strips features with the smallest weights/coefficients.

3. Embedded Methods

Embedded methods perform feature selection during model training natively. Examples include LASO (L1 Regularization) which drives coefficients to zero ($\min ||y - Xw||^2_2 + \lambda||w||_1$) and Decision Tree Gini importance metrics.

```
      import pandas as pd from sklearn.feature_selection import RFE, SelectKBest, chi2 from
sklearn.linear_model import LogisticRegression, LassoCV # 1. Filter Method: Select top K
features via Chi-Square selector = SelectKBest(score_func=chi2, k=5) X_filter =
selector.fit_transform(X, y) # 2. Wrapper Method: Recursive Feature Elimination model =
LogisticRegression(max_iter=1000) rfe = RFE(estimator=model, n_features_to_select=5)
X_wrapper = rfe.fit_transform(X, y) # 3. Embedded Method: LASO Feature Selection lasso =
LassoCV(cv=5).fit(X, y) selected_features = X.columns[lasso.coef_ != 0]
```

4. User Retention Analytics

User retention analytics applies data extraction and behavioral feature engineering to measure how effectively a product keeps users engaged over time. It forms the backbone of customer churn modeling, lifetime value (LTV) estimation, and product growth strategies.

Core Metrics in Retention Analytics

- **N-Day Retention:** The percentage of users who return to the application on exactly the N-th day after their initial sign-up event: `Retention(Day N) = Users Active on Day N / Users Registered on Day 0`.
- **Unbounded Retention:** Percentage of users who return on Day N or any day thereafter.
- **Cohort Analysis:** Grouping users by sign-up date/channel and tracking their activity matrix over time to isolate product improvements from seasonal noise.
- **Churn Rate:** Percentage of active subscribers who discontinue service during a defined time window: `Churn = Lost Users / Total Start Users`.

> **Feature Engineering for Churn Prediction**
>
> Features extracted from event logs include: recency (days since last login), frequency (number of actions per week), duration (average session length), and feature depth (number of unique tools utilized).

5. Mathematical Foundations: Entropy & Information Gain

Information theory provides the objective mathematical framework for quantifying impurity and information content in data splits for decision tree learning.

1. Shannon Entropy

Entropy measures the level of disorder, uncertainty, or impurity within a set of class labels S. For a dataset with C target classes:

$$H(S) = - \sum_{i=1}^{C} p_i \log_2(p_i)$$

Where p_i is the proportion of samples belonging to class i. For a pure dataset (all samples belong to one class), $H(S) = 0$. For a balanced binary split, $H(S) = 1.0$.

2. Information Gain (IG)

Information Gain measures the expected reduction in entropy achieved by partitioning dataset S using feature A:

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} (|S_v| / |S|) \times H(S_v)$$

> **Step-by-Step Calculation Example**
>
> Consider a set S of 14 samples: 9 Positive, 5 Negative. $H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$.
>
> Splitting on feature A yields S_{left} (6 Pos, 2 Neg; $H = 0.811$) and S_{right} (3 Pos, 3 Neg; $H = 1.000$).
>
> Weighted Entropy = $(8/14)(0.811) + (6/14)(1.000) = 0.892$.
>
> Information Gain = $0.940 - 0.892 = 0.048$ bits.

6. Decision Tree Algorithms & Splitting Criteria

Decision trees recursively partition feature space into orthogonal axis-aligned hyperplanes to maximize class purity in leaf nodes.

Splitting Metrics Comparison

- **Gini Impurity (CART):** Measures probability of misclassifying a randomly chosen element: $Gini(S) = 1 - \sum p_i^2$. Computationally faster than entropy as it avoids logarithms.
- **Gain Ratio (C4.5):** Normalizes Information Gain by Split Information to prevent bias toward features with many distinct categorical values: $GainRatio = IG(S, A) / SplitInfo(A)$.

Algorithm	Split Metric	Tree Structure	Feature Handling
ID3	Information Gain	Multi-way splits	Categorical features only
C4.5	Gain Ratio	Multi-way splits	Categorical & Continuous
CART	Gini Impurity / MSE	Binary splits only	Classification & Regression

Overfitting & Tree Pruning

Unconstrained trees grow until every leaf is pure, creating complex boundaries that overfit noise. **Pre-pruning** stops tree growth early using depth/min-sample thresholds. **Post-pruning** grows a full tree and strips subtrees based on Cost-Complexity Pruning: $R_\alpha(T) = R(T) + \alpha|T|$.

7. Random Forests & Ensemble Learning

Random Forest is an ensemble learning method that constructs a multitude of decorrelated decision trees during training and outputs the mode class (classification) or mean prediction (regression) of the individual trees.

Key Architectural Principles

- **Bootstrap Aggregation (Bagging):** Each tree is trained on a random sample drawn with replacement from the training dataset, containing roughly 63.2% of unique rows.
- **Random Feature Subspace Sampling:** At every node split, the algorithm evaluates only a random subset of features `m = √p` (where `p` is total features). This decorrelates individual trees.
- **Out-of-Bag (OOB) Error Estimation:** The ~36.8% of rows left out of each tree's bootstrap sample serve as an implicit validation set, enabling unbiased error estimation during training without separate cross-validation folds.

> **Gini Feature Importance**
>
> Feature importance is calculated as the sum over all trees of the reduction in Gini Impurity brought by splits on that variable, weighted by the proportion of samples reaching each split node.

8. The Dimensionality Problem (Curse of Dimensionality)

The Curse of Dimensionality refers to various phenomena that arise when analyzing and organizing data in high-dimensional spaces (often hundreds or thousands of features) that do not occur in low-dimensional settings.

Key Theoretical Consequences

- **Extreme Data Sparsity:** As dimensions increase, the volume of the space grows exponentially ($V \propto s^d$). Available data points become isolated, requiring exponentially more data to maintain density.
- **Distance Concentration Effect:** In high dimensions, the distance between any two points converges to the same value. The relative difference between the distance to the nearest point (D_{min}) and farthest point (D_{max}) approaches zero: $\lim_{d \to \infty} (D_{max} - D_{min}) / D_{min} \to 0$.
- **Degradation of Distance Metrics:** Standard metrics like Euclidean distance lose contrast and discriminating power, crippling k-NN, k-means clustering, and spatial indexing algorithms.

> **Mitigation Strategies**
>
> Overcoming high dimensionality requires explicit feature selection, regularization techniques, or projection onto low-dimensional linear/non-linear manifolds using SVD and PCA.

9. Single Value Decomposition (SVD)

Single Value Decomposition is a fundamental linear algebra matrix factorization theorem that decomposes any real $m \times n$ matrix A into three constituent matrices.

Mathematical Formulation

$A = U \Sigma V^T$

- **U (Left Singular Vectors):** An $m \times m$ orthogonal matrix ($U^TU = I$) whose columns are eigenvectors of AA^T, representing data row patterns.
- **Σ (Singular Values):** An $m \times n$ diagonal matrix with non-negative real singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ sorted in descending order, representing component variance scales.
- **V (Right Singular Vectors):** An $n \times n$ orthogonal matrix ($V^TV = I$) whose columns are eigenvectors of A^TA, representing column feature spaces.

Truncated SVD for Dimensionality Reduction

By retaining only the top k singular values ($A_k = U_k \Sigma_k V_k^T$), we construct the best rank-k approximation of matrix A according to the Eckart-Young theorem. Widely applied in Latent Semantic Analysis (LSA) and recommender systems.

10. Principal Component Analysis (PCA) & Review

PCA is an unsupervised linear transformation technique that identifies orthogonal axes (principal components) maximizing dataset variance.

Algorithmic Steps

1. Standardize input feature matrix X to mean zero and unit variance.
2. Compute the $n \times n$ Covariance Matrix: $\Sigma = (1/m)\ X^T\ X$.
3. Perform Eigen-decomposition: $\Sigma\ v_i = \lambda_i\ v_i$ to extract Eigenvectors v_i and Eigenvalues λ_i.
4. Sort eigenvectors by eigenvalues descending; choose top k components to form projection matrix W.
5. Project features to lower dimensional space: $Z = X\ W$.

```
      import numpy as np import pandas as pd from sklearn.decomposition import PCA from
sklearn.preprocessing import StandardScaler # Standardize data X_std =
StandardScaler().fit_transform(X) # Fit PCA retaining 95% of explained variance pca =
PCA(n_components=0.95) X_pca = pca.fit_transform(X_std) print(f"Original features:
{X.shape[1]}") print(f"Reduced components: {pca.n_components_}") print(f"Explained Variance
Ratio: {pca.explained_variance_ratio_}")
```

Module 2 Exam Review Questions

- Explain why Information Gain is biased toward high-cardinality features and how Gain Ratio corrects it.
- Derive the relationship between PCA principal components and SVD right singular vectors.